

3.4. MÁQUINAS DE ESTADOS FINITOS

Os circuitos sequenciais síncronos podem ser desenhados nas formas apresentadas na [Figura 3.22](#). Estas formas são chamadas de máquinas de estados finitos (FSM). Eles obtiveram este nome porque um circuito com k registros pode estar num de um número finito (2^k) de estados únicos. Uma FSM tem entradas M , N saídas, e k bits de estado. Ele também recebe um relógio e, opcionalmente, um sinal de *reset*. Uma FSM é composta por dois blocos de lógica combinatória, lógica do estado seguinte e lógica de saída, e um registro que armazena o estado. Em cada flanco do relógio, a FSM avança para o próximo estado, que foi calculado com base no estado atual e nas entradas. Existem duas classes gerais de máquinas de estados finitos, caracterizadas pelas suas especificações funcionais. Nas máquinas de Moore, as saídas dependem apenas do estado atual da máquina. Nas máquinas de Mealy, as saídas dependem tanto do estado atual e como das entradas atuais. As máquinas de estados finitos fornecem uma forma sistemática de projetar circuitos sequenciais síncronos dada uma especificação funcional. Este método irá ser explicado no restante desta secção, começando com um exemplo.

3.4.1. Projeto Exemplo de FSM

Para ilustrar a concepção das FSM, considere o problema de inventar um controlador para um semáforo num cruzamento movimentado no campus. Os estudantes de engenharia estão a deambular entre os dormitórios e os laboratórios na Av. Académica. Eles estão ocupados lendo sobre as FSM nos seus livros favoritos e não estão olhando para onde estão indo. Os jogadores de futebol estão a mover-se apressadamente entre os campos de atletismo e sala de jantar em Blvd. Bravado. Eles estão jogando à bola para trás e para frente e não estão também olhando para onde estão indo. Vários ferimentos graves já ocorreram no cruzamento destas duas ruas, e o decano dos estudantes pede a Ben Bitdiddle para instalar um semáforo antes que haja mortes.

As máquinas de Moore e de Mealy receberam o nome dos seus inventores, investigadores que desenvolveram a teoria de autómatos, os fundamentos matemáticos das máquinas de estado, na Bell Labs. Edward F. Moore (1925-2003), não deve ser confundido com o fundador da Intel Gordon Moore, publicou seu artigo seminal, *Gedanken-experiments on Sequential Machines* em 1956. Em seguida, tornou-se professor de matemática e ciência da computação na Universidade de Wisconsin.

George H. Mealy publicou um método de síntese de circuitos sequenciais em 1955. Posteriormente, ele escreveu o primeiro sistema operativo da Bell Labs para o computador IBM 704. Mais tarde, ele se juntou à Universidade de Harvard.

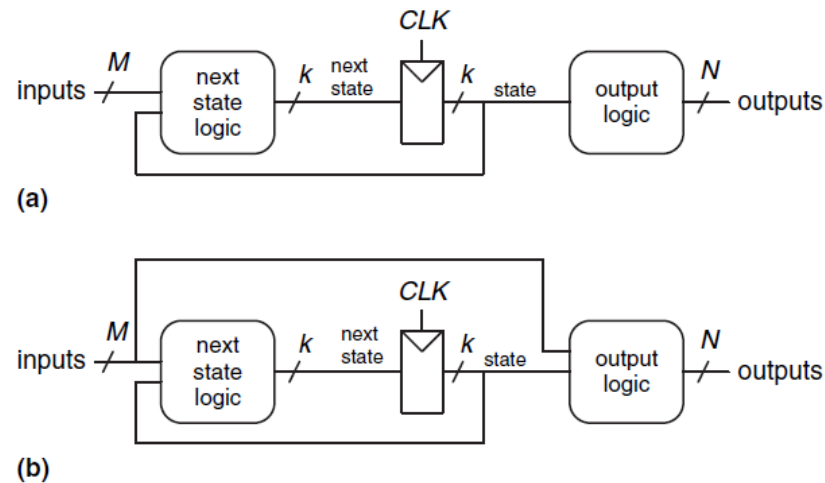


Figura 3.22 Máquinas de estados finitos: (a) A máquina de Moore, (b) máquina de Mealy

Ben decide resolver o problema com uma FSM. Ele instala dois sensores de tráfego, T_A e T_B , na Av. Acadêmica e no Blvd. Bravado, respectivamente. Cada sensor indica TRUE se os alunos estão presentes e FALSE se a rua está vazia. Ele também instala dois semáforos, L_A e L_B , para controlar o tráfego. Cada luz recebe entradas digitais para especificar se ela deve estar verde, amarela ou vermelha. Por isso, a sua FSM tem duas entradas, T_A e T_B , e duas saídas, L_A e L_B . O cruzamento com luzes e sensores são mostrados na [Figura 3.23](#). Ben fornece um relógio com um período de 5 segundos. Em cada pulso de relógio (flanco ascendente), as luzes podem mudar de acordo com os sensores de tráfego. Ele também fornece um botão de *reset* para que os técnicos possam colocar o controlador num estado inicial conhecido ao ligá-lo. A [Figura 3.24](#) mostra uma vista da caixa preta da máquina de estados.

O próximo passo de Ben é esboçar o diagrama de transição de estado, mostrado na [Figura 3.25](#), para indicar todos os possíveis estados do sistema e as transições entre esses estados. Quando o sistema é reiniciado, as luzes estão verde na Av. Acadêmica e vermelho em Blvd. Bravado. A cada 5 segundos, o controlador examina o padrão de tráfego e decide o que fazer a seguir. Enquanto o tráfego está presente na Av. Acadêmica, as luzes não mudam. Quando já não existe tráfego na Av. Acadêmica, a luz da Av.

Académica torna-se amarela durante 5 segundos antes de ficar vermelha, e a luz do Blvd. Bravado fica verde. Da mesma forma, a luz da Blvd. Bravado permanece verde enquanto o tráfego está presente na avenida, em seguida, torna-se amarela e, eventualmente, vermelha.

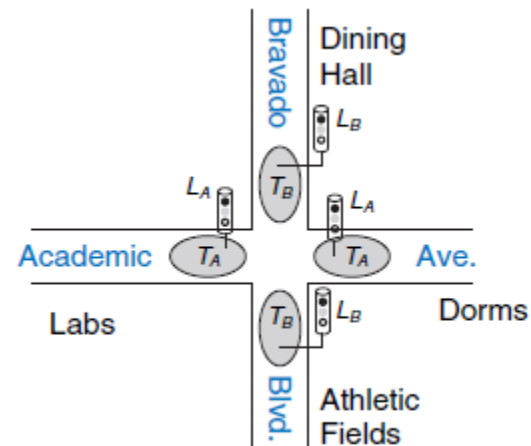


Figura 3.23 Mapa do campus

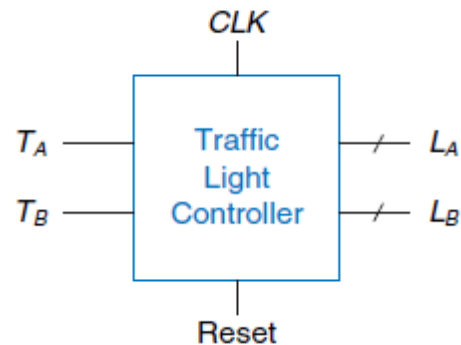


Figura 3.24 Vista de caixa preta da máquina de estados finitos

Observe que os estados são designados como S_0 , S_1 , etc. As versões subscritas, S_0 , S_1 , etc., referem-se aos bits de estado.

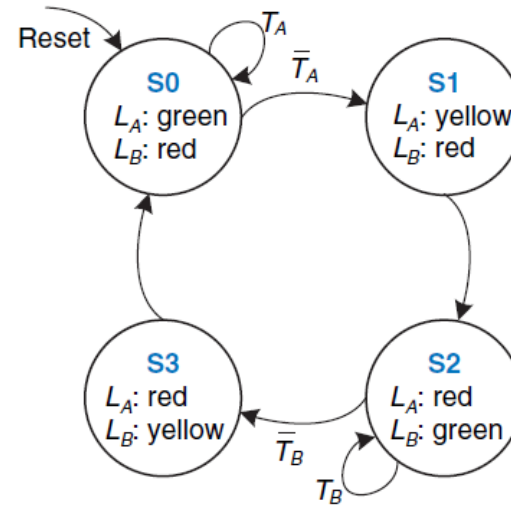


Figura 3.25 Diagrama da transição de estados

Num diagrama de transição de estado, os círculos representam os estados e os arcos representam as transições entre estados. As transições ocorrem no flanco ascendente do relógio; que não é representado no diagrama, porque ele está sempre presente num circuito sequencial síncrono. Além disso, o relógio controla quando devem ocorrer as transições, enquanto que o gráfico indica que transições ocorrem. O arco ilustrado apontando do espaço exterior para o estado S_0 indica que o sistema deve entrar nesse estado após *reset*, independentemente do estado anterior em que se encontrava. Se um estado tem vários arcos deixando-o, os arcos são rotulados para mostrar que entrada aciona cada transição. Por exemplo, quando no estado S_0 , o sistema irá permanecer nesse estado se T_A é TRUE e passar para S_1 se T_A é FALSE. Se um estado tem um único arco a deixá-lo, a transição ocorre sempre, independentemente das entradas. Por exemplo, quando no estado S_1 , o sistema irá sempre transitar para S_2 . Os valores que as saídas têm enquanto num determinado estado são indicados no estado. Por exemplo, enquanto no estado S_2 , L_A é vermelha e L_B é verde.

O Ben reescreve o diagrama de transição de estado como uma tabela de transição de estado (Tabela 3.1 que indica, para cada estado e entrada, qual será o próximo estado, S' . Note que a tabela usa o

símbolo (X) (*don't care*) sempre que o próximo estado não depende de uma entrada em particular. Além disso, note que *Reset* é omitido da tabela. Em vez disso, usamos *resettable flip-flops* que sempre vão para o estado S_0 quando reinicia, independentes das entradas.

O diagrama de transição de estado é abstrato usando estados designadas $\{S_0, S_1, S_2, S_3\}$ e saídas designadas {vermelha, amarela, verde}. Para construir um circuito real, aos estados e às saídas devem ser atribuídas codificações binárias. Ben escolhe as codificações dadas nas [Tabela 3.2](#) e [Tabela 3.3](#). Cada estado e cada saída são codificados com dois bits: $S_{1:0}$, $L_{A1:0}$, e $L_{B1:0}$.

Tabela 3.1 Tabela de transição de estados

Current State S	Inputs T_A T_B		Next State S'
S_0	0	X	S_1
S_0	1	X	S_0
S_1	X	X	S_2
S_2	X	0	S_3
S_2	X	1	S_2
S_3	X	X	S_0

Tabela 3.2 Codificação de estados

State	Encoding $S_{1:0}$
S_0	00
S_1	01
S_2	10
S_3	11

Tabela 3.3 Codificação das saídas

Output	Encoding $L_{1:0}$
green	00
yellow	01
red	10

O Ben actualiza a tabela de transição do estado para usar estas codificações binárias, como mostra a [Tabela 3.4](#). A tabela de transição de estado revista é uma tabela verdade especificando o próximo estado lógico. Ela define o próximo estado, S' , como uma função do estado actual, S , e das entradas.

A partir desta tabela, é fácil serem lidas as equações Booleanas para o próximo estado na forma de soma de produtos.

$$\begin{aligned}
 S'_1 &= \bar{S}_1 S_0 + S_1 \bar{S}_0 \bar{T}_B + S_1 \bar{S}_0 T_B \\
 S'_0 &= \bar{S}_1 S_0 \bar{T}_A + S_1 \bar{S}_0 \bar{T}_B
 \end{aligned}
 \tag{3.1}$$

As equações podem ser simplificadas usando mapas de Karnaugh, mas muitas vezes fazê-lo por inspeção torna-se mais fácil. Por exemplo, os termos T_B e $\bar{T}_B$ na equação S'_1 são claramente redundantes. Assim S'_1 reduz-se a uma operação XOR. Equação 3.2 dá as equações simplificadas de próximo estados.

Tabela 3.4 Codificação binária das transições de estado

Current State S_1 S_0		Inputs T_A T_B		Next State S'_1 S'_0	
0	0	0	X	0	1
0	0	1	X	0	0
0	1	X	X	1	0
1	0	X	0	1	1
1	0	X	1	1	0
1	1	X	X	0	0

Tabela 3.5 Tabela de saída

Current State		Outputs			
S_1	S_0	L_{A1}	L_{A0}	L_{B1}	L_{B0}
0	0	0	0	1	0
0	1	0	1	1	0
1	0	1	0	0	0
1	1	1	0	0	1

$$\begin{aligned} S'_1 &= S_1 \oplus S_0 \\ S'_0 &= \bar{S}_1 \bar{S}_0 \bar{T}_A + S_1 \bar{S}_0 \bar{T}_B \end{aligned} \quad (3.2)$$

Da mesma forma, Ben escreve uma tabela de saída (Tabela 3.5), indicando, para cada estado, qual deve ser a saída nesse estado. Novamente, é fácil de serem lidas e simplificar as equações Booleanas para as saídas. Por exemplo, observar que L_{A1} é TRUE apenas nas linhas onde S_1 é TRUE.

$$\begin{aligned} L_A &= S_1 \\ L_{A0} &= \bar{S}_1 S_0 \\ L_A &= \bar{S}_1 \\ L_{B0} &= S_1 S_0 \end{aligned} \quad (3.3)$$

Finalmente, Ben esboça a sua FSM de Moore na forma da Figura 3.22 (a). Em primeiro lugar, ele desenha o registro de estado de dois bits, como mostrado na Figura 3.26 (a). Em cada flanco do relógio, o estado de registro copia o próximo estado, $S'_{1:0}$, para se tornar o estado $S_{1:0}$. O registro de estado recebe um *reset* síncrono ou assíncrono para inicializar a FSM no arranque. Em seguida, ele desenha a lógica do próximo estado, com base na equação 3.2, que calcula o próximo estado a partir do estado atual e das entradas, como mostrado na Figura 3.26 (b). Finalmente, ele desenha a lógica de saída, com base na equação 3.3, que calcula as saídas a partir do estado atual, como mostrado na Figura 3.26 (c).

A Figura 3.27 mostra um diagrama temporal que ilustra o controlador de semáforo passando por uma sequência de estados. O diagrama mostra CLK , $Reset$, as entradas T_A e T_B , o próximo estado S' , o estado S , e as saídas L_A e L_B . As setas indicam causalidade; por exemplo, alterando o estado faz com que

Este esquema usa algumas portas AND com negações nas entradas. Elas podem ser construídas com portas AND e inversores de entrada, com portas NOR e inversores para as entradas não negadas ou com alguma outra combinação de portas. A melhor escolha depende da tecnologia de implementação em particular.

as saídas se alterem, e mudando as entradas faz com que o próximo estado a mude. As linhas a tracejado indicam os flancos ascendentes do CLK , quando o estado muda.

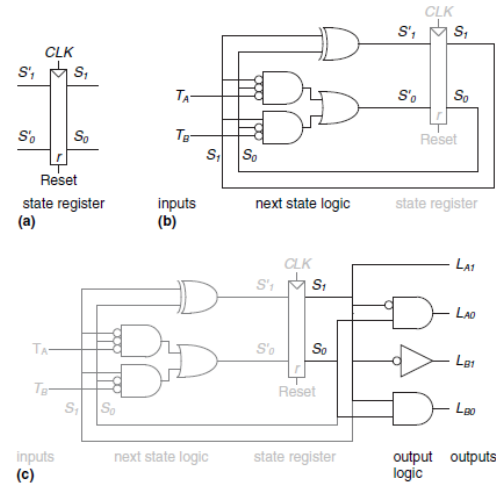


Figura 3.26 Circuito da máquina de estados para o controlador de semáforo

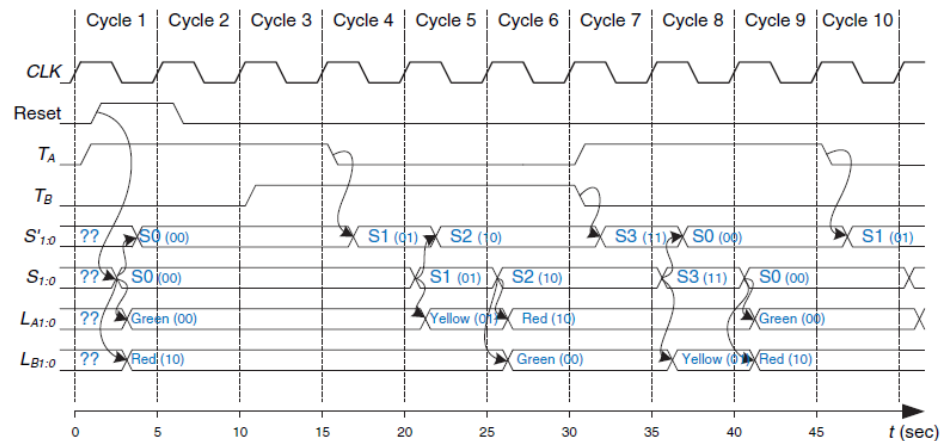


Figura 3.27 Diagrama temporal do controlador de semáforo

O relógio tem um período de 5 segundos, assim os semáforos mudam no máximo uma vez a cada 5 segundos. Quando a máquina de estados finitos é ligada pela primeira vez, o seu estado é desconhecido, como indicado pelos pontos de interrogação. Portanto, o sistema deve ser redefinido de modo a colocá-lo num estado conhecido. Neste diagrama temporal, S é redefinido imediatamente para S_0 , indicando que os *resettable flip-flops* assíncronos estão sendo usados. No estado S_0 , a luz L_A está verde e a luz L_B está vermelha.

Neste exemplo, o tráfego chega imediatamente na Av. Acadêmica. Portanto, o controlador permanece no estado S_0 , mantendo L_A verde mesmo que o tráfego chegue no Blvd. Bravado e inicia a espera. Após 15 segundos, todo o tráfego passou na Av. Acadêmica e T_A cai. No relógio seguinte, o controlador segue para o estado S_1 , transformando L_A em amarelo. Em mais 5 segundos, o controlador prossegue para o estado S_2 em que L_A fica vermelha e L_B fica verde. O controlador espera no estado S_2 até que todo o tráfego na Blvd. Bravado passe. Ele então transita para S_3 , fazendo L_B amarelo. Depois de 5 segundos, o controlador entra no estado S_0 , transformando L_B em vermelho e L_A em verde. O processo repete indefinidamente.

3.4.2. Codificação de Estados

No exemplo anterior, as codificações de estado e de saída foram selecionadas arbitrariamente. Uma escolha diferente teria resultado num circuito diferente. Uma pergunta natural é como determinar a codificação que produz o circuito com o menor número de portas lógicas ou o atraso de propagação mais curto. Infelizmente, não existe uma maneira simples de encontrar a melhor codificação, exceto tentar todas as possibilidades, o que é inviável quando o número de estados é elevado. No entanto, muitas vezes é possível escolher uma boa codificação por inspeção, para que os estados ou as saídas relacionadas partilhem bits. Ferramentas *computer-aided* (CAD) são também boas na busca do conjunto de possíveis codificações e selecionando um razoável.

Uma decisão importante na codificação de estados é a escolha entre codificação binária e a codificação *one-hot*. Com codificação binária, como foi usada no exemplo do controlador de semáforo, cada estado está representado como um número binário. Como os K números binários a poderem ser representados pelos $\log_2 K$ bits, um sistema com K estados apenas necessita de $\log_2 K$ bits de estado.

Apesar dos melhores esforços de Ben, os alunos não prestam atenção aos semáforos e as colisões continuam a ocorrer. O decano dos estudantes pergunta-lhe a ele e à Alyssa para projetar uma catapulta para lançar os estudantes de engenharia diretamente para os telhados dos seus dormitórios através das janelas abertas do laboratório, ignorando o cruzamento problemático. Mas isso é assunto para outro livro.



Na codificação *one-hot*, um bit separado de estado é usado para cada estado. Chama-se *one-hot* porque apenas um bit está "quente" ou verdadeiro em qualquer momento. Por exemplo, uma FSM codificada em *one-hot* com três estados teria codificações de estados de 001, 010 e 100. Cada bit de estado é armazenado num *flip-flop*, então uma codificação *one-hot* requer mais *flip-flops* do que a codificação binária. No entanto, com a codificação *one-hot*, a lógica de próximo estado e saída é muitas vezes mais simples, de modo que são necessários menos portas. A melhor escolha de codificação depende do FSM específico.

Exemplo 3.6 CODIFICAÇÃO DOS ESTADOS DE UMA FSM

Um contador de divisão-por- N possui uma saída e não tem entradas. A saída Y está HIGH para N ciclos de relógio. Em outras palavras, a saída divide a frequência do relógio por N . O diagrama de transição de estado e a forma de onda para um contador divisor-por-3 é mostrado na Figura 3.28. Esboce o circuito para este contador usando codificação de estado binário e *one-hot*.

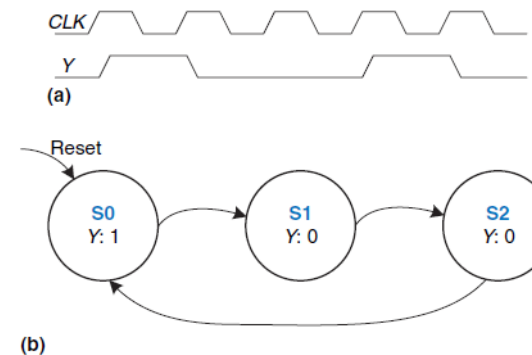


Figura 3.28 Contador divide-por-3 (a), forma de onda e (b) diagrama de transição de estado

Solução: As Tabela 3.6 e Tabela 3.7 mostram as transições de estado e saída antes da codificação.

A Tabela 3.8 compara a codificação binária e *one-hot* para os três estados.

A codificação binária usa dois bits de estado. Usando esta codificação, a tabela de transição de estado é mostrada na Tabela 3.9. Note-se que não existem entradas; o próximo estado depende apenas do

estado atual. A tabela de saída é deixada como um exercício para o leitor. As equações de próximo estado e de saída são:

$$\begin{aligned} S'_1 &= \bar{S}_1 S_0 \\ S'_0 &= \bar{S}_1 \bar{S}_0 \end{aligned} \quad (3.4)$$

$$Y = \bar{S}_1 \bar{S}_0 \quad (3.5)$$

Tabela 3.6 Tabela de transição de estado do contador divide-por-3

Current State	Next State
S0	S1
S1	S2
S2	S0

Tabela 3.7 Tabela de saída do contador divide-por-3

Current State	Output
S0	1
S1	0
S2	0

A codificação *one-hot* usa três bits de estado. A tabela de transição de estado para esta codificação é mostrada na [Tabela 3.10](#) e a tabela de saída é novamente deixada como um exercício para o leitor. As equações de próximo estado e de saída são as seguintes:

$$\begin{aligned} S'_2 &= S_1 \\ S'_1 &= S_0 \\ S'_0 &= S_2 \end{aligned} \quad (3.6)$$

$$Y = S_0 \quad (3.7)$$

A Figura 3.29 mostra os esquemas para cada um destes projetos. Note-se que o hardware para o projeto da codificada binária poderia ser otimizado para compartilhar a mesma porta entre Y e S'_0 . Observe também que a codificação *one-hot* requer *flip-flops* com (*s*) *settable* e (*r*) *resettable* para inicializar a máquina para S_0 no arranque. A melhor escolha de implementação depende do custo relativo das portas e dos *flip-flops*, mas o projeto *one-hot* geralmente é preferível para este exemplo específico.

Tabela 3.8 Codificações e binária e *one-hot* para contador dividir-por-3

State	One-Hot Encoding			Binary Encoding	
	S_2	S_1	S_0	S_1	S_0
S0	0	0	1	0	0
S1	0	1	0	0	1
S2	1	0	0	1	0

Tabela 3.9 Tabela de transição de estado com codificação binária

Current State		Next State	
S_1	S_0	S'_1	S'_0
0	0	0	1
0	1	1	0
1	0	0	0

Tabela 3.10 Tabela de transição de estado com codificação *one-hot*

Current State			Next State		
S_2	S_1	S_0	S'_2	S'_1	S'_0
0	0	1	0	1	0
0	1	0	1	0	0
1	0	0	0	0	1

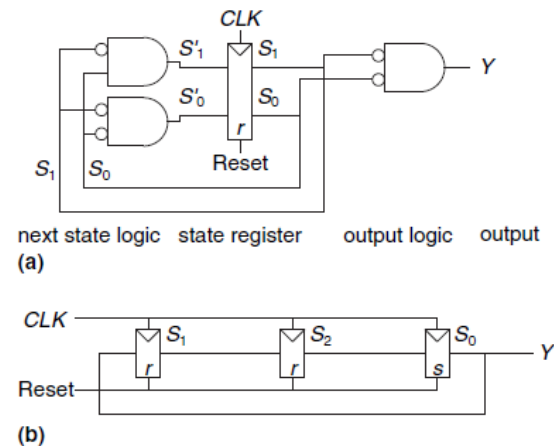


Figura 3.29 Contador divide-por-3 (a), forma de onda e (b) diagrama de transição de estado

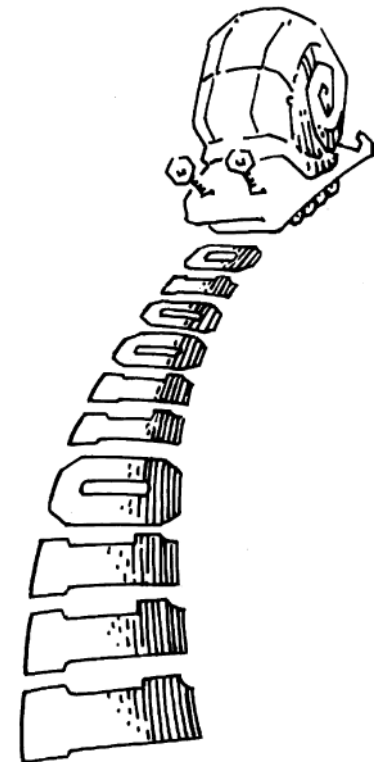
3.4.3. Máquinas de Moore e de Mealey

Até agora, temos mostrado exemplos de máquinas de Moore, na qual a saída depende apenas do estado do sistema. Assim, nos diagramas de transição de estado para máquinas de Moore, as saídas são rotuladas nos círculos. Lembre-se que as máquinas de Mealy são muito parecidas com as máquinas de Moore, mas as saídas podem depender tanto das entradas, como do estado atual. Assim, nos diagramas de transição de estado para máquinas de Mealy, as saídas são colocadas nos arcos em vez de nos círculos. O bloco de lógica combinatória que calcula as saídas usa o estado atual e as entradas, como foi mostrado na Figura 3.22 (b).

Exemplo 3.7 MÁQUINA DE MOORE VS MÁQUINA DE MEALY

Alyssa P. Hacker é dono de um caracol robótico de estimação com um cérebro FSM. O caracol rasteja a partir da esquerda para a direita ao longo de uma fita de papel contendo uma sequência de 1 e 0. Em cada ciclo de *clock*, o caracol rasteja para o próximo bit. O caracol sorri quando os dois últimos bits ao longo dos quais se arrastou são, da esquerda para a direita, 01. Projete a FSM para calcular quando o caracol deve sorrir. A entrada A é a parte debaixo das antenas do caracol. A saída Y é TRUE quando o

Uma maneira fácil de lembrar a diferença entre os dois tipos de máquinas de estados finitos é que uma máquina de Moore geralmente tem mais estados do que uma máquina de Mealy para um determinado problema.



caracol sorri. Compare os projetos das máquinas de estado de Moore e Mealy. Esboce um diagrama temporal para cada máquina que mostra a entrada, os estados e a saída quando o caracol de Alyssa se arrasta ao longo da sequência 0100110111.

Solução: A máquina de Moore requer três estados, como mostrado na [Figura 3.30 \(a\)](#). Considere que o diagrama de transição de estado está correto. Em particular, por que é que há um arco de S2 para S1 quando a entrada está a 0?

Em comparação, a máquina Mealy requer apenas dois estados, como mostrado na [Figura 3.30 \(b\)](#) (arco é identificado como A/Y. A é o valor da entrada que causa essa transição, e Y é a saída correspondente).

As [Tabela 3.11](#) e [Tabela 3.12](#) mostram a transição de estado e tabela de saída para a máquina de Moore. A máquina de Moore requer pelo menos dois bits de estado. Considere o uso da codificação binária de estado: S0 = 00, S1 = 01, e S2 = 10. As [Tabela 3.13](#) e [Tabela 3.14](#) reescrevem as tabelas de transição de estado e de saída com estas codificações.

A partir dessas tabelas, encontramos as seguintes equações de estado e de saída por inspeção. Note-se que estas equações são simplificadas fazendo uso do fato de que o estado 11 não existe. Deste modo, o próximo estado e saída correspondente para o estado que não existe são *don't care* (não mostrado nas tabelas). Usamos os *don't care* para minimizar as nossas equações.

$$S'_1 = S_0 A \quad (3.8)$$

$$S'_0 = \bar{A}$$

$$Y = S_1 \quad (3.9)$$

A [Tabela 3.15](#) mostra a transição de estado e de saída para a máquina de Mealy. A máquina de Mealy requer apenas um bit de estado. Considere o uso de uma codificação de estado binária: S0 = 0 e S1 = 1. A [Tabela 3.16](#) reescreve a transição de estado e de saída com estas codificações.

A partir dessas tabelas, encontramos as seguintes equações de estado e de saída por inspeção.

$$S'_0 = \bar{A} \quad (3.10)$$

$$Y = S_0A \quad (3.11)$$

Os esquemas das máquinas de Moore e de Mealy são mostrados na [Figura 3.31](#). Os diagramas temporais para cada máquina são mostrados na [Figura 3.32](#). As duas máquinas seguem uma sequência diferente de estados. Além disso, a saída da máquina Mealy sobe um ciclo mais cedo porque responde à entrada em vez de esperar a mudança de estado. Se a saída da máquina de Mealy fosse atrasada através de um *flip-flop*, ela iria coincidir com a saída da máquina de Moore. Quando escolher o seu estilo do projeto da FSM, o leitor deve considerar quando quer que as suas saídas respondam.

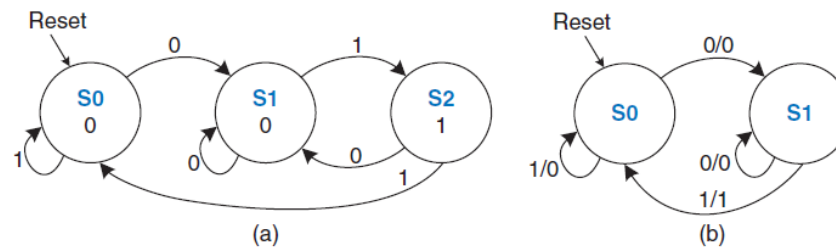


Figura 3.30 Diagramas de transição de estado da FSM: (a) A máquina de Moore, (b) máquina de Mealy

Tabela 3.11 Tabela de transição de estado da máquina de Moore

Current State <i>S</i>	Input <i>A</i>	Next State <i>S'</i>
S0	0	S1
S0	1	S0
S1	0	S1
S1	1	S2
S2	0	S1
S2	1	S0

Tabela 3.12 Tabela de saída da máquina de Moore

Current State S	Output Y
S0	0
S1	0
S2	1

Tabela 3.13 Tabela de transição de estado da máquina de Moore com codificações de estado

Current State $S_1 \quad S_0$		Input A	Next State $S'_1 \quad S'_0$	
0	0	0	0	1
0	0	1	0	0
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0

Tabela 3.14 Tabela de saída da máquina de Moore com codificações de estado

Current State $S_1 \quad S_0$		Output Y
0	0	0
0	1	0
1	0	1

Tabela 3.15 Tabela de transição de estado e saída da máquina de Mealy

Current State S	Input A	Next State S'	Output Y
S0	0	S1	0
S0	1	S0	0
S1	0	S1	0
S1	1	S0	1

Tabela 3.16 Tabela de transição de estado da máquina de Mealy com codificações de estado

Current State S_0	Input A	Next State S'_0	Output Y
0	0	1	0
0	1	0	0
1	0	1	0
1	1	0	1

3.4.4. Factoring Máquinas de Estado

O projeto de FSM complexas é frequentemente mais fácil se elas forem divididas em várias máquinas de estado mais simples que interagem de tal modo que as saídas de algumas máquinas são as entradas de outras. Esta aplicação da hierarquia e modularidade é chamada de *factoring* de máquinas de estado.

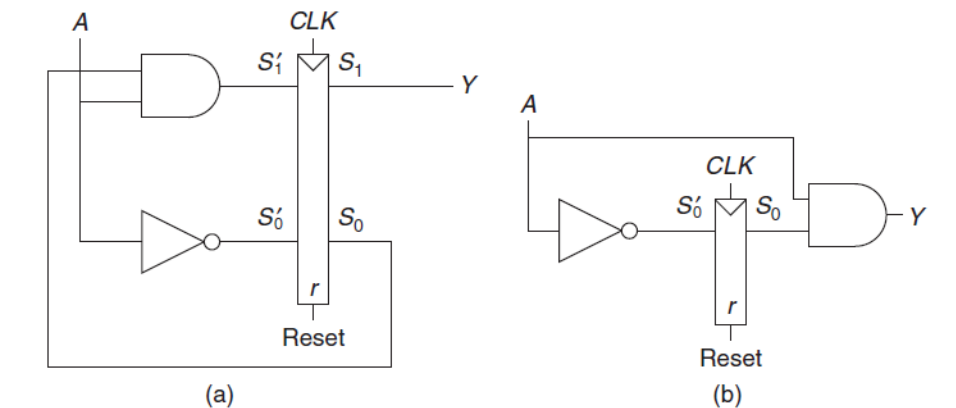


Figura 3.31 Esquemas de FSM para máquinas (a) Moore e (b) de Mealy

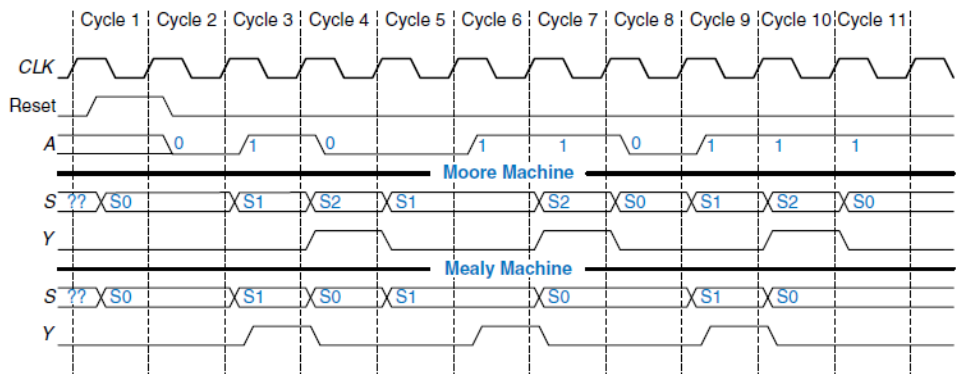


Figura 3.32 Diagramas temporais para as máquinas de Moore e de Mealy

Exemplo 3.8: MÁQUINAS DE ESTADO *UNFACTORED* E *FACTORED*

Modifique o controlador de semáforo da Seção 3.4.1 para ter um modo de desfile, o que mantém a luz verde para o Boulevard Bravado enquanto os espectadores e a banda marcham para jogos de futebol em grupos dispersos. O controlador recebe mais duas entradas: P e R . Ativando P por pelo menos um ciclo entra-se no modo de desfile. Ativando R durante pelo menos um ciclo sai-se do modo de desfile. Quando em modo de desfile, o controlador procede através de sua sequência normal até L_B ficar verde, em seguida, permanece nesse estado com L_B verde até que o modo desfile termine.

Em primeiro lugar, esboce um diagrama de transição de estado para uma única FSM, como mostrado na Figura 3.33 (a). Em seguida, esboce os diagramas de transição de estado para duas FSM interagindo, como mostrado na Figura 3.33 (b). O *Modo FSM* ativa a saída M quando se está no modo de desfile. A *Lights FSM* controla as luzes com base em M e nos sensores de tráfego, T_A e T_B .

Solução: A Figura 3.34 (a) mostra o desenho da EFM. Os estados S0 a S3 lidam com o modo normal. Os estados S4 a S7 lidam com o modo desfile. As duas metades do diagrama são quase idênticas, mas no modo de parada, a FSM permanece no estado S6 com a luz verde no Blvd. Bravado. As entradas P e R controlam o movimento entre estas duas metades. O projeto de FSM é confuso e entediante. A Figura 3.34 (b) mostra o projeto *factored* da FSM. O *Modo FSM* tem dois estados para controlar se as luzes estão no modo normal ou no modo desfile. A *Ligth FSM* é modificada para permanecer em S2, enquanto M é TRUE.

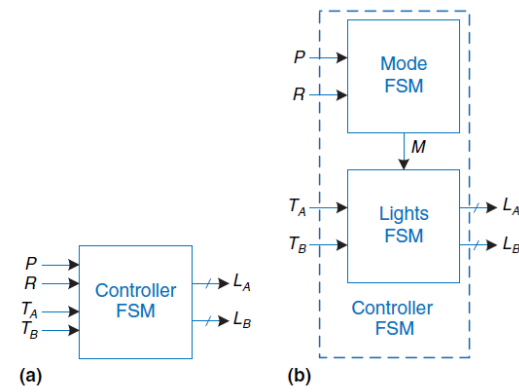


Figura 3.33 Projetos para a FSM controlador de tráfego modificada a) único e (b) *factored*

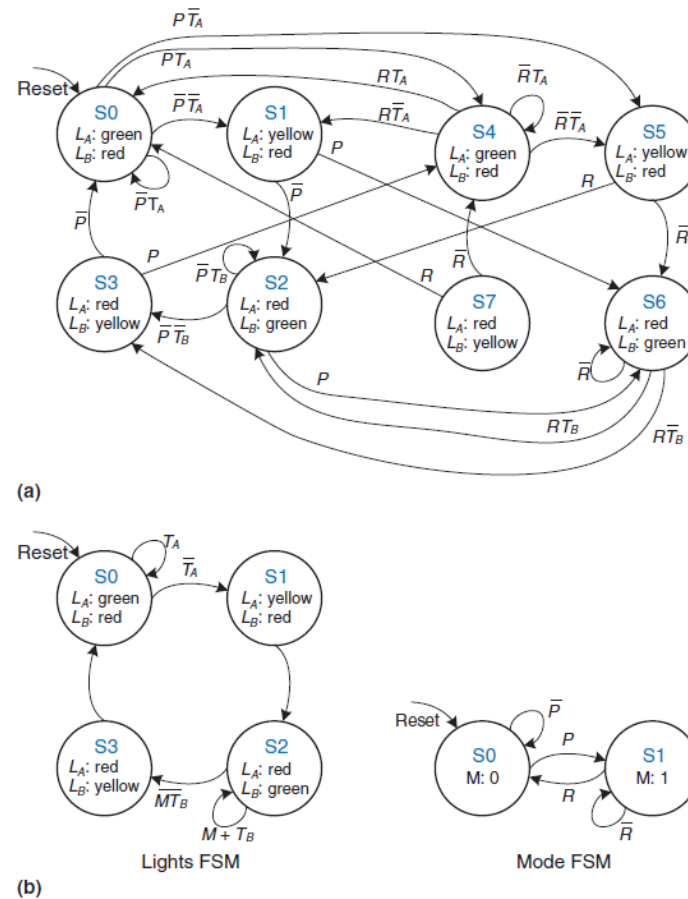


Figura 3.34 Diagrama de transição de estados: (a) não *factored* (b) *factored*

3.4.5. Obtendo uma FSM de um Esquema

A derivação do diagrama de transição de estado a partir de um esquema segue quase o processo inverso do projeto de uma FSM. Este processo pode ser necessário, por exemplo, quando se pega num projeto com documentada incompleta ou caso seja necessário fazer engenharia reversa do sistema de outra pessoa.

- Examine o circuito, ativando as entradas, saídas, e bits de estado.
- Escreva as equações dos próximos estado e de saída.
- Crie as tabelas dos próximos estado e de saída.
- Reduza a tabela de próximo estado para eliminar estados inacessíveis.
- Atribua a cada combinação de bits de estado válidos um nome.
- Reescreva a tabela de próximo estado e de saída com nomes de estado.
- Desenhe o diagrama de transição de estado.
- Escreva por palavras o que a FSM faz.

Na etapa final, deve ter o cuidado de descrever sucintamente o objetivo global e função da FSM - Não basta voltar a descrever cada transição do diagrama de transição de estado.

Exemplo 3.9 OBTER UMA FSM DO SEU CIRCUITO

Alyssa P. Hacker chega em casa, mas o teclado do seu acesso foi religado e o seu código antigo não funciona. Um pedaço de papel está colado a ele tal como se mostra no diagrama do circuito da [Figura 3.35](#). Alyssa pensa que o circuito poderá ser uma máquina de estados finitos e decide derivar o diagrama de transição de estado para ver a ajuda a abrir a porta.

Solução: Alyssa começa por examinar o circuito. A entrada é $A_{1:0}$ e a saída é *Unlock*. Os bits de estado já estão marcados na [Figura 3.35](#). Esta é uma máquina de Moore, porque a saída depende apenas dos bits de estado. Do circuito, ela retira diretamente as equações de próximo estado e de saída:

$$\begin{aligned} S'_1 &= S_0 \overline{A_1} A_0 \\ S'_0 &= \overline{S_1} \overline{S_0} A_1 A_0 \\ \text{Unlock} &= S_1 \end{aligned} \tag{3.12}$$

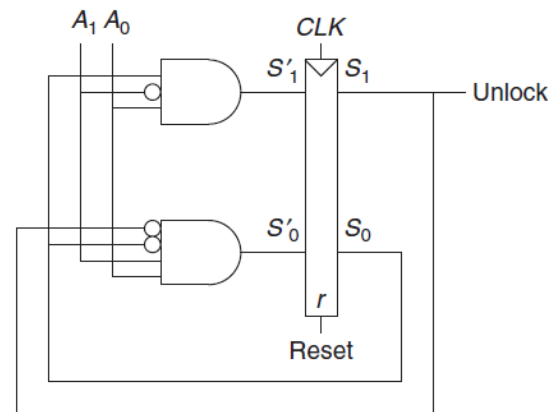


Figura 3.35 Circuito da FSM encontrada no Exemplo 3.9

Em seguida, ela escreve as equações das tabelas de próximo estado e de saída, como mostrado nas [Tabela 3.17](#) e [Tabela 3.18](#), colocando primeiro 1 nas posições das tabelas como indicado pela Equação 3.12. Ela coloca em seguida 0 nos restantes lugares.

Alyssa reduz a tabela, removendo os estados não utilizados e combinando linhas *don't cares*. O estado $S_{1:0} = 11$ nunca é listado como sendo um possível próximo estado na [Tabela 3.17](#), então as linhas com este estado atual são removidas. Para o estado atual $S_{1:0} = 10$, o próximo estado é sempre $S_{1:0} = 00$, independentemente das entradas, por isso *don't care* são inseridos para as entradas. As tabelas reduzidas são mostradas nas [Tabela 3.19](#) e [Tabela 3.20](#).

Ela atribui nomes a cada combinação de bits de estado: $S0$ é $S_{1:0} = 00$, $S1$ é $S_{1:0} = 01$ e $S2$ é $S_{1:0} = 10$. As [Tabela 3.21](#) e [Tabela 3.22](#) mostram as tabelas de próximo estado e de saída com os nomes de estado.

Tabela 3.17 Tabela de próximo estado derivada do circuito na Figura 3.36

Current State		Input		Next State	
S_1	S_0	A_1	A_0	S'_1	S'_0
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	0
0	0	1	1	0	1
0	1	0	0	0	0
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	0	1	0	0
1	0	1	0	0	0
1	0	1	1	0	0
1	1	0	0	0	0
1	1	0	1	1	0
1	1	1	0	0	0
1	1	1	1	0	0

Tabela 3.18 Tabela da saída proveniente do circuito da Figura 3.35

Current State		Output
S_1	S_0	<i>Unlock</i>
0	0	0
0	1	0
1	0	1
1	1	1

Tabela 3.19 Tabela simplificada do próximo estado

Current State S_1 S_0		Input A_1 A_0		Next State S'_1 S'_0	
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	0
0	0	1	1	0	1
0	1	0	0	0	0
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	1	0	0
1	0	X	X	0	0

Tabela 3.20 Tabela simplificada da saída

Current State S_1 S_0		Output <i>Unlock</i>
0	0	0
0	1	0
1	0	1

Tabela 3.21 Tabela simbólica do próximo estado

Current State <i>S</i>	Input <i>A</i>	Next State <i>S'</i>
S0	0	S0
S0	1	S0
S0	2	S0
S0	3	S1
S1	0	S0
S1	1	S2
S1	2	S0
S1	3	S0
S2	X	S0

Tabela 3.22 Tabela simbólica da saída

Current State <i>S</i>	Output <i>Unlock</i>
S0	0
S1	0
S2	1

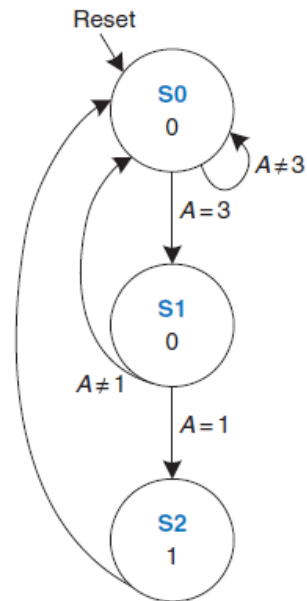


Figura 3.36 Diagrama de transição de estado da FSM encontrada do Exemplo 3.9

Alyssa escreve o diagrama de transição de estado mostrado na [Figura 3.36](#) usando as [Tabela 3.21](#) e [Tabela 3.22](#). Por inspeção, ela pode ver que a máquina de estados finitos destranca a porta só depois de detectar um valor de entrada, $A_{1:0}$, de 3 seguido por um valor de entrada de 1. A porta é, então, novamente bloqueada. Alyssa tenta este código no teclado da porta e esta abre-se!

3.4.6. Revisão das FSM

As máquinas de estados finitos são uma forma poderosa de projetar sistematicamente circuitos sequenciais a partir de uma especificação escrita. Use o seguinte procedimento para projetar um FSM:

- Identifique as entradas e as saídas.
- Esboce um diagrama de transição de estado.

- Para uma máquina de Moore:
 - Faça uma tabela de transição de estado.
 - Faça uma tabela de saída.
- Para uma máquina Mealy:
 - Escreva uma tabela combinada das transições de estado e de saída.
- Selecione a codificação dos estados – a sua seleção afeta o projeto do hardware.
- Escreva as equações Booleanas para a lógica do próximo estado e de saída.
- Esboce o esquema de circuito.

Vamos usar repetidamente as FSM para projetar sistemas digitais complexos ao longo deste livro.

3.5. TEMPORIZAÇÃO DA LÓGICA SEQUENCIAL

Lembre-se que um *flip-flop* copia a entrada D para a saída Q no flanco ascendente do relógio. Este processo é chamado de amostragem (*sampling*) de D na borda ascendente do relógio. Se D está estável a 0 ou a 1 quando o relógio sobe, este comportamento é claramente definido. Mas o que acontece se D está mudando ao mesmo tempo que o relógio sobe?

Este problema é semelhante ao que se coloca a uma câmara ao tirar uma fotografia. Imagine que fotografa um sapo pulando de uma almofada de lírio para o lago. Se o leitor tirar a foto antes do salto, irá ver um sapo numa almofada de lírio. Se o leitor tirar a foto depois do salto, irá ver ondulações na água. Mas se o leitor tirar a foto, assim que o sapo pula, o leitor poderá ver uma imagem do sapo esborratada que se estende desde a almofada de lírio até à água. Uma câmara é caracterizada pelo seu tempo de abertura, durante o qual o objeto tem de permanecer parado para se capturar uma imagem nítida. Do mesmo modo, um elemento sequencial tem um tempo de abertura em torno do flanco do relógio, durante o qual a entrada deve ser estável para que o *flip-flop* possa produzir uma saída bem definida.



Projeto Digital e Arquitetura de Computadores

By David Money Harris & Sarah L. Harris



Morgan Kaufman
© English 2nd Edition 2013



Esta tradução Portuguesa foi patrocinada
pela Imagination Technologies
www.imgtec.com
community.imgtec.com/university